Aksum University

Faculty Of Computing Technology

Department Of Computer Science

Group Assignment: Design And Analysis of Algorithm

No	NAME	ID-No
1	Weldearegay Gebrey	Aku-1602148
2	Eyerusalem Teklay	Aku-1602682
3	Shewit Legese	Aku-1602069

N-Queens Problem Solver Using Backtracking with Pruning

❖ Introduction

The **N-Queens Problem** is a well-known problem in computer science that focuses on placing **N queens on an N × N chessboard** such that no two queens attack each other. A queen in chess can attack horizontally, vertically, and diagonally, so the challenge is to ensure none of these conflicts occur.

This problem is important because it demonstrates key concepts such as:

- **Backtracking algorithms**
- **Constraint Satisfaction Problems (CSP)**
- Efficient search techniques

In this project, we implement an optimized solution using **backtracking with pruning**, which reduces unnecessary computations and improves performance.

★ Problem Statement

The goal is to arrange **N queens on an N × N chessboard** so that:

- Each row contains exactly one queen
- Each column contains exactly one queen
- No two queens lie on the same diagonal

For example:

- When **N = 8**, there are **92 valid solutions**
- The complexity increases rapidly as N grows

This problem becomes computationally challenging for larger values of N.

✓ Objectives

The main objectives of this project are:

- To implement the N-Queens problem using **backtracking**
- To improve efficiency using a **pruning (bounding) technique**
- To use a **1D array representation** for simplicity
- To calculate:
    - Total number of solutions
    - Execution time
    - Nodes explored
- To analyze the algorithm's **time and space complexity**
- To understand real-world applications of constraint-based problems

♦ Methodology

We use the **backtracking algorithm with pruning**, which works as follows:

1. Start placing queens from the first row
2. For each row, try placing a queen in every column
3. Before placing, check if the position is **safe**
4. If safe:
    a. Place the queen
    b. Move to the next row
5. If not safe:
    a. Skip that position (prune the branch)
6. If all queens are placed successfully:
    a. A valid solution is found
7. If no valid position is found:
    a. Backtrack to the previous row

This approach avoids exploring invalid configurations early.

❖ Data Representation

We use a **1D array** to represent the chessboard:

```
queens[row] = column
```

Example:

```
[1, 5, 8, 6, 3, 7, 2, 4]
```

This means:

- Row 0 → Column 1
- Row 1 → Column 5
- Row 2 → Column 8
- and so on

Advantages:

- Reduces memory usage
- Simplifies implementation
- Makes conflict checking easier

★ Bounding Function (Pruning Technique)

The **bounding function (`isSafe`)** ensures that no conflicts occur before placing a queen.

It checks:

★ ***Column Conflict***

If any previous queen is in the same column:

```
prevCol == col
```

♦ ***Diagonal Conflict***

If queens lie on the same diagonal:

```
|row - prevRow| == |col - prevCol|
```

Importance:

- Prevents invalid placements early
- Reduces the search space
- Improves performance significantly

❖ Results and Observations

The algorithm was tested for different values of N:

N = 8

- Solutions: **92**
- Nodes explored: **15,720**
- Time: **~16 ms**

N = 10

- Solutions: **724**
- Nodes explored: **348,150**
- Time: **~25 ms**

N = 12

- Solutions: **14,200**
- Nodes explored: **10,103,868**
- Time: **~259 ms**

Observations:

- Number of solutions increases rapidly
- Nodes explored grow exponentially
- Pruning reduces runtime significantly

❖ Complexity Analysis

Time Complexity

- Worst case: **O(N!)**
- Without pruning: **$O(N^n)$** (very inefficient)

Backtracking reduces the number of possibilities by eliminating invalid solutions early.

Space Complexity

- **O(N)**
- Due to recursion depth and array storage

✓ Conclusion on Complexity

Although the worst-case complexity is high, pruning makes the algorithm practical for moderate values of N.

❖ Real-World Applications and Conclusion

★ Applications

The N-Queens problem is not just theoretical; it has real-world relevance:

1. **Constraint Satisfaction Problems (CSP)**
    a. Assign values while satisfying constraints
2. **Antenna Placement**
    a. Avoid signal interference
3. **Task Scheduling**
    a. Assign tasks without conflicts
4. **VLSI Circuit Design**
    a. Place components without interference

❑ Conclusion

This project demonstrates how **backtracking combined with pruning** can efficiently solve complex combinatorial problems like the N-Queens problem.

Key takeaways:

- Backtracking explores all possibilities systematically
- Pruning reduces unnecessary computation
- The solution is scalable but computationally intensive for large N

★ Future Improvements (Optional Section)

- Use **bitmask optimization** for faster execution
- Implement **parallel processing**
- Add a **graphical user interface (GUI)**
- Solve for larger values of N (e.g., N > 12)